COLLEGE OF ENGINEERING, GUINDY
ANNA UNIVERSITY, CHENNAI – 25

Railway Digital Twin with Wildlife Safety

An Educational Simulation Integrating Godot 4, MQTT,
a Trusted Node.js Control Plane and ESP32 Edge Sensing

Submitted in partial fulfillment of the course
"Embedded Systems and Internet of Things"

Submitted By

Team: 14

Register Number	Name
2023103546	Kathir Kalidass B
2023103597	Malini R
2023103600	Parthasarathi S
2023103714	Paril T

October 2026

Abstract

Train–animal collisions remain a persistent operational risk on forested and rural railway corridors while conventional level-crossing protection offers the same single mechanism for every kind of intruder. This project develops an educational *Railway Digital Twin with Wildlife Safety*: a complete, runnable miniature of a modern wildlife-protected railway line in which the same event can be observed simultaneously from four cooperating surfaces — a three-dimensional twin simulation, a web operations dashboard, a trusted backend control plane, and an ESP32 edge controller, each communicating over an authenticated IoT message bus.

The system is built as a staged software stack. **Godot 4.7** renders a two-track "stadium" railway world (UP and DOWN loops of 1253.98 m and 1228.85 m around a 250 m straight-sided footprint) with an island station, interlocked crossovers, a wildlife level crossing with boom barriers, carved river with road and railway bridges, catenary, dynamic signal masts and Libre-TrainSim-derived Blender train, station, deer and elephant assets. **Node.js 22+** with Express and SQLite is the trusted control plane: the sole publisher of railway commands, state snapshots and alerts, enforcing bcrypt/JWT authentication, role-based access control (viewer/operator/admin), command-envelope validation, anti-replay protection through a persistent SQLite registry, and a full audit trail. **Eclipse Mosquitto 2** brokers the machine traffic on MQTT TCP 1883/WebSocket 9001 with per-client credentials and least-privilege topic ACLs. A **React 18/Vite** dashboard presents live state, per-block signal aspects, per-train speed control, meet scheduling, wrong-line routing and hazard evidence badges. An **ESP32 (Arduino/PlatformIO)** node carries fail-safe field inputs — wildlife and sensor-fault pulls, protective outputs, debounce, SNTP-gated clock truth and non-blocking telemetry. A **FastAPI ML microservice** using torchvision's SSDLite MobileNetV3 detector confirms animal sightings from twin-camera frames behind a two-frame evidence policy.

The wildlife protection chain implements a staged escalation NONE → SUSPECTED (30 km/h limit) → CONFIRMED (0 km/h, STOP/RED) driven by trackside sensor huts, onboard simulated cameras and ESP32 edge reports, with backend-authoritative hazard fusion, recovery-latched resets and restrictive fail-safe behaviour on any loss of

communication. Every subsystem is verified by an automated gate — 42/42 Godot suites, 141/141 backend tests, 8/8 ML tests, 16/16 ESP32 native tests and a clean web build — plus live end-to-end scenarios including a chance-based meet scheduling, an interlocked wrong-line run and the two-frame camera confirmation chain end-to-end on the live stack. The system is emphatically a classroom simulation and not certified railway signalling equipment.

Keywords: Digital Twin, Railway Safety, Wildlife, MQTT, IoT, ESP32, Godot Engine, Node.js, React, Machine Learning

Contents

Abstract **ii**

1 Introduction **1**

1.1 Background 1

1.2 Objectives 2

1.3 Scope and safety boundary 2

1.4 Phased implementation outline 3

1.5 Organisation of the report 4

2 System Requirements and Phases **5**

2.1 Functional requirements 5

2.2 Non-functional requirements 7

2.3 Hardware requirements 8

2.4 Software requirements 9

2.5 Phase-to-gate mapping 10

3 System Architecture and Workflow **11**

3.1 Architecture overview 11

3.2 Data flow and message contract 12

3.3 Backend internal architecture 13

3.4 Failure handling and fail-safety 14

3.5 Topic contract summary 14

List of Figures

3.1 System block diagram: one trusted control plane, many clients. 11

3.2 End-to-end workflow: control (steps 1–4) and wildlife observation (steps 5–9) as one closed loop. 13

List of Tables

2.1 Functional requirements and delivery phase 5
2.2 Non-functional requirements 7
2.3 Hardware requirements 8
2.4 Software requirements 9
2.5 Delivery phases vs verification evidence 10

Chapter 1

Introduction

1.1 Background

Indian Railways reserves operate through large tracts of forest and open grassland in which scheduled and unscheduled train paths intersect with animal movement corridors. Elephants, deer and other wildlife cross railway formation at grade, and collisions remain a persistent operational and ecological risk. Conventional protection — signage, whistle zones and manual watchmen — is sparse by design, and a single fixed level-crossing alarm cannot adapt on its own to *which* animal is ahead, *where* it is relative to a moving train, or *how much* braking distance the train still has.

Modern practice addresses this with two converging ideas:

- **Digital twins:** a live virtual replica of a physical asset (here, a forest railway line with its trains, signals and crossing) that stays synchronised with trusted state and lets operators observe, rehearse and reason about operations.
- **Embedded IoT sensing:** low-cost edge devices (here, an ESP32 with sensors, protective outputs and a fail-safe state machine) at the trackside, publishing machine observations over an authenticated message bus rather than treated independently.

This project, developed as an educational simulation by Team 14 for the course *Embedded Systems and Internet of Things*, brings both ideas together in a system small enough to run on one laptop: a three-dimensional digital twin of a forest railway loop, a trusted backend control plane, a role-aware web dashboard, an authenticated MQTT broker, and an ESP32 edge controller — all exchanging sensor observations, authority commands, state snapshots and alerts over well-defined topics.

1.2 Objectives

The objectives of the project are:

1. Build a runnable **3D digital twin** of a double-track forest railway — UP and DOWN loops, an island station with interlocking crossovers, a wildlife level crossing with boom barriers, a carved river with road and railway bridges, catenary, dynamic signals and an LTS-derived Blender asset fleet — in Godot 4.

2. Construct a **trusted control plane** that owns all railway authority: a two-train double-track fleet model, per-block signal aspects, a fixed-timestep simulation loop, a station-crossover interlocking, chance-based meet scheduling, an operator-requestable interlocked wrong-line run, and a wildlife-hazard state machine — served over authenticated REST and MQTT.

3. Enforce **security end to end**: authenticated users with role-based access control, an authenticated MQTT broker with per-client topic ACLs, command-envelope validation with timestamp freshness and anti-replay protection, and a complete audit trail.

4. Implement **wildlife sensing and staged protection**: a NONE → SUSPECTED → CONFIRMED escalation driven by trackside sensor huts, an onboard simulated camera detector with analytic occlusion and a quadratic stopping-distance model, a back-end camera ingest pipeline with two-frame evidence confirmation, and ML-assisted animal detection through a FastAPI microservice.

5. Implement an **ESP32 edge node** with restrictive fail-safe outputs, debounced local inputs, SNTP-gated clock truth, non-blocking telemetry and edge-triggered wildlife alerts — validated natively and in the Wokwi cloud simulator.

6. Provide a **role-aware dashboard** for viewer, operator and admin users with live state, per-block aspects, per-train speed control, route/meet/wrong-line panels and hazard evidence badges.

7. Verify the whole stack continuously with an automated multi-surface test gate and documented live end-to-end scenarios.

1.3 Scope and safety boundary

Two scope statements are as important as the features themselves:

- **Educational simulation only.** The system is a classroom digital-twin simulation, *not* railway safety equipment. It must never be connected to real signalling, trains or operational infrastructure. Simulated "stopping distance" is a visualization estimate only and does not model certified train dynamics, gradients, adhesion or real braking curves.
- **Physical hardware status.** The ESP32 path was verified through the Wokwi cloud simulator (via a Pinggy tunnel to the local authenticated broker). The physical-hardware checklist items in the Stage D plan remain *not run*: real instruments were not connected. The report is explicit wherever this applies.

Within those boundaries, every subsystem — backend, twin, dashboard, firmware and ML service — is fully implemented and gated by automated tests (Chapter **??**, Appendix **??**).

1.4 Phased implementation outline

The system was built in recorded, gated stages (dates from `docs/porting/acceptance.md`):

- **Stage A (backend foundation)** — Express + JWT/RBAC, SQLite audit and anti-replay registry, two-zone state machine, backend-authoritative MQTT, broker auth and ACLs (2026-10-18/19).
- **Stage A2 (block signalling)** — per-block aspects and retained signals (2026-10-19).
- **Stage B (dashboard)** — login, role-aware REST controls, WebSocket live updates, audit feed (2026-10-19).
- **Stage C (Godot twin)** — vendored MQTT bridge, procedural world, train/wildlife animation, brakes, signals, cameras; porting Libre-TrainSim (LTS) Blender assets (2026-10-19/20).
- **Stage D (ESP32 / Wokwi)** — per-client credentials, device-scoped topics, fail-safe outputs, cloud simulation through a tunnel (2026-10-19/20).
- **Stage E (integrated delivery)** — Docker/environment setup, scenario scripts and the formal acceptance record (2026-10-22).
- **Stage 2A–2G (double-track phase)** — a two-train UP/DOWN fleet, ten blocks with aspects, station island and crossovers (2A), the double-track world (2B–2C),

backend lives and camera fleet (2A–2D), meet/pass scenarios (2E, 2G), and distributed wildlife sensing with an onboard detector (2F) (2026-10-21/22).

- **Stadium evolution — Phases 1–3** — both loops reshaped as one 250 m straight-sided shared stadium footprint with opposite operational chainage (2026-10-22), an interlocked operator wrong-line run with an opposing-train hold (2026-10-23), and the staged camera/ML/near-animal confirmation chain with two-frame evidence (2026-10-23).

1.5 Organisation of the report

Chapter 2 states requirements. Chapter 3 describes the system architecture and end-to-end workflow. Chapter 4 documents tools and technology. Chapter 5 covers hardware, peripherals and the simulation stack. Chapter 6 details the digital twin world. Chapter 7 documents the dashboards. Chapter 8 lists every implemented feature and function with its verification evidence. Chapter 9 presents results, limitation and conclusion. Appendices collect the access-control matrix, the complete test-gate record and run instructions. Reproduced simulation screenshots appear in Figures **??**–**??** and the figures referenced from Chapters 5–8.

Chapter 2

System Requirements and Phases

2.1 Functional requirements

Table 2.1: Functional requirements and delivery phase

S.No	Requirement	Phase
FR-1	Authenticate users (bcrypt passwords, JWT sessions) with three roles: viewer (monitor), operator (control), admin (audit inspection).	A
FR-2	Validate every command envelope against schema with ±5 min timestamp freshness and reject replays through a persistent SQLite `processed_requests` registry.	A
FR-3	Record an audit row (actor, role, zone, action, result, timestamp) for every control attempt — accepted, rejected or forbidden.	A
FR-4	On MQTT disconnect apply the restrictive `COMM_LOST` fail-safe; on reconnect publish the retained state snapshot to reconcile clients.	A
FR-5	Be the sole publisher of commands, state snapshots, alerts and signals on MQTT with per-client topic ACLs.	A
FR-6	Reject a reset while a wildlife hazard or sensor fault is active (operator must clear and then explicitly reset).	A
FR-7	Model a two-train (T1 UP, T3 DOWN) double-track fleet with ten blocks, per-block signal aspects, occupancy safety and a fixed-timestep simulation loop.	2A

Table 2.1: (continued)

S.No	Requirement	Phase
FR-8	Provide a station-crossover interlock: one route at a time, hazard/occupancy checks, 409 on conflicts, automatic release.	2A, 2G
FR-9	Run chance-based meet scheduling from a seeded PRNG: hold the slower train at the station, execute the crossover as a proper route, reproduce history for identical seeds.	2G
FR-10	Let an operator request an interlocked wrong-line run (UP→DOWN at crossover A, return at B) while the opposing train is held automatically inside braking distance; phases APPROACH → CROSS_ENTRY → WRONG_LINE → CROSS_EXIT → CLEARANCE → DONE.	Stadium P2
FR-11	Operate distributed trackside wildlife huts — UP-S0..UP-S4 and DOWN-S0..DOWN-S4, i.e. ten huts with 75 m overlapping radius — publishing observations used as *trackside* evidence.	2F
FR-12	Provide onboard simulated-ML camera detection (forward cone vision, ±25°/10°, 200 m range, analytic occlusion against terrain, trunks and bridge decks) as *onboard* evidence with zero-latency local braking.	2F
FR-13	Accept camera frames over authenticated HTTP (bearer token, `X-Captured-At` freshness, duplicate-id suppression), validate blocks and line, run the ML detector, and confirm a hazard only on *two* frames.	Stadium P3
FR-14	Implement staged hazard escalation: trackside-only suspicion caps the approaching train at 30 km/h and shows YELLOW; camera confirmation stops the train (0 km/h) and shows RED; clearing never downgrades a confirmed block; reset is required after hazard clear.	Stadium P3
FR-15	Deliver ESP32 edge alerts (`WILDLIFE_NEAR`, `WILDLIFE_DETECTED`, `WILDLIFE_CLEARED`) to the edge node and require operator re-arm (RESET) through the dashboard.	Stadium P3

Table 2.1: (continued)

S.No	Requirement	Phase
FR-16	Render the twin: stadium double-track world, island station, interlocked crossovers, two-track level crossing, river with bridges, catenary, dynamic signals, barriers, cameras and HUD.	C, 2B–2E, Stadium P1
FR-17	Provide the dashboard: login, live state via WebSocket, train cards with speed control, block-aspect diagram, evidence badges, meet card, wrong-line panel, camera health.	B, 2E–2G, P3
FR-18	Publish ESP32 telemetry with SNTP-validated UTC time; reject stale, malformed, duplicate or wrong-zone telemetry at the backend and mark clock-invalid telemetry before it is sent.	D

2.2 Non-functional requirements

Table 2.2: Non-functional requirements

S.No	Requirement	Where enforced
NFR-1	**Security — least privilege:** every MQTT client authenticates; per-client ACLs allow the dashboard read-only access, Godot telemetry-publish-only on its zones, and the backend full control topics.	`mosquitto/acl_file`, broker AC
NFR-2	**Fail-safety:** every output is restrictive at startup, on communication loss, on fresh-context loss, and for active local hazard/fault inputs; local inputs cannot be cleared remotely.	backend state machine, ESP32 c
NFR-3	**Anti-replay:** accepted REST control IDs/timestamps persists in SQLite; stale, duplicated or schema-invalid envelopes are rejected.	`command_tracker.js`/SQLite
NFR-4	**Auditability:** every control or route attempt recorded in SQLite regardless of outcome; admin-only inspection.	`audit.js`

Table 2.2: (continued)

S.No	Requirement	Where enforced
NFR-5	**Reproducibility:** seeded PRNG (meet scheduler and wildlife director) gives identical histories for identical seeds; same command must produce the same state.	`rng.js`, `meets.js`
NFR-6	**Performance:** backend snapshot publish loop of 2 Hz; train-speed updates smooth at the twin; live state and camera ingest stay within documented limits (camera ingest >5 fps rate-limited, 413 on oversize).	`loop.js`, camera ingest
NFR-7	**Testability:** every surface gated by automated tests; the gate must exit non-zero on any failure and be re-runnable headless.	`verify-all.sh`, test suites
NFR-8	**Educational honesty:** stopping distance is a visualization estimate; not certified signalling; the system never touches real railway infrastructure.	README, presentation
NFR-9	**Compose-neutral developer path:** one script `dev-up.sh` to stand up the stack, reset the demo, generate broker credentials.	scripts/

2.3 Hardware requirements

Table 2.3: Hardware requirements

Item	Specification
Development host	x86-64 PC with Linux or WSL2; mouse/keyboard for the twin. GPU useful but not required (Mesa llvmpipe software rendering is supported for headless verification)
Docker engine	Docker Engine with Compose v2; no "docker desktop" required
ESP32 devkit v1	Optional for the edge-node path (equivalently simulated on Wokwi); Wi-Fi capable, MCP UART 115 200

Sensors/indicators	Green/red LEDs, buzzer, pushbuttons (wildlife, fault), SSD1306 OLED, servo (all simulated in Wokwi); telemetry needs network reachability to the broker
Cameras	None physical; twin camera feeds are simulated render outputs consumed by the ML microservice

2.4 Software requirements

Table 2.4: Software requirements

Layer	**Tool / version**
OS	Ubuntu 24.04 LTS (WSL2 permissible)
Container runtime	Docker Engine + Compose; images `eclipse-mosquitto:2`, Node base image
Control plane	Node.js 22+ (runtime 24 of the verification host), Express 4.21, better-sqlite3 13, bcryptjs 3, jsonwebtoken 9, mqtt.js 5.16
Broker	Eclipse Mosquitto 2 — TCP 1883, WebSocket 9001, `allow_anonymous false`
Dashboard	React 18.3.1, Vite 6, TypeScript 5.7, `lucide-react`
Twin engine	Godot 4.7.2 (GL compatibility for headless render)
Firmware	PlatformIO (`espressif32`, `arduinoframework`), PubSubClient 2.8, ArduinoJson 7, ESP32Servo, Adafruit SSD1306/GFX
Edge simulation	Wokwi CLI v0.29.0; tunnel via Pinggy raw TCP
ML microservice	Python, FastAPI 0.135.1, uvicorn, torch 2.10.0, torchvision 0.25.0 (SSDLite320 MobileNetV3, COCO animal classes), Pillow, NumPy
Quality	Node built-in test runner (backend), Godot test harness (42 suites), Pytest (ML), PlatformIO Unity (ESP32 native), Vite/TypeScript build check

2.5 Phase-to-gate mapping

Table 2.5: Delivery phases vs verification evidence

Phase	Deliverable	Evidence
A / A2	Trusted backend, two-zone and later block signalling	`state.test.js`, `signalling.test.js`, `broker_acl.test.js`
B	Authenticated role-aware dashboard	Rest API tests, web build
C	Godot twin with LTS-derived assets	42 Godot suites incl. `test_real_assets`
D	ESP32 fail-safe edge node	16 native Unity cases + Wokwi scenario
E	Integrated stack + formal acceptance record	`docs/porting/acceptance.md`
2A–2C	Double-track fleet, world, crossovers	`signalling/routes/meets` suites, Godot track registry suites
2E–2G	Meet/pass and chance-based meets with interlocking	`test_meet_scenario`, `meets.test.js`, `rng.test.js`
2F	Distributed wildlife sensing + onboard detector + braking model	`test_sensor_huts`, `test_onboard_detector`, `test_onboard_braking`
Stadium P1–P3	Shared stadium geometry, wrong-line run, camera/ML/near-animal chain	`test_stadium_track`, `wrong_line*.test.js`, `near_animal.test.js`, `camera.test.js`

Chapter 3

System Architecture and Workflow

3.1 Architecture overview

The system is organised around one principle: **one trusted control plane, many observing/edge clients**. The Node.js backend is the single authority for train commands, block aspects, routes, meets, hazards and alerts; every other component either publishes observations (Godot twin, ESP32) or consumes authoritative state (dashboard, twin, ESP32). Figure 3.1 shows the five subsystems and their interfaces.

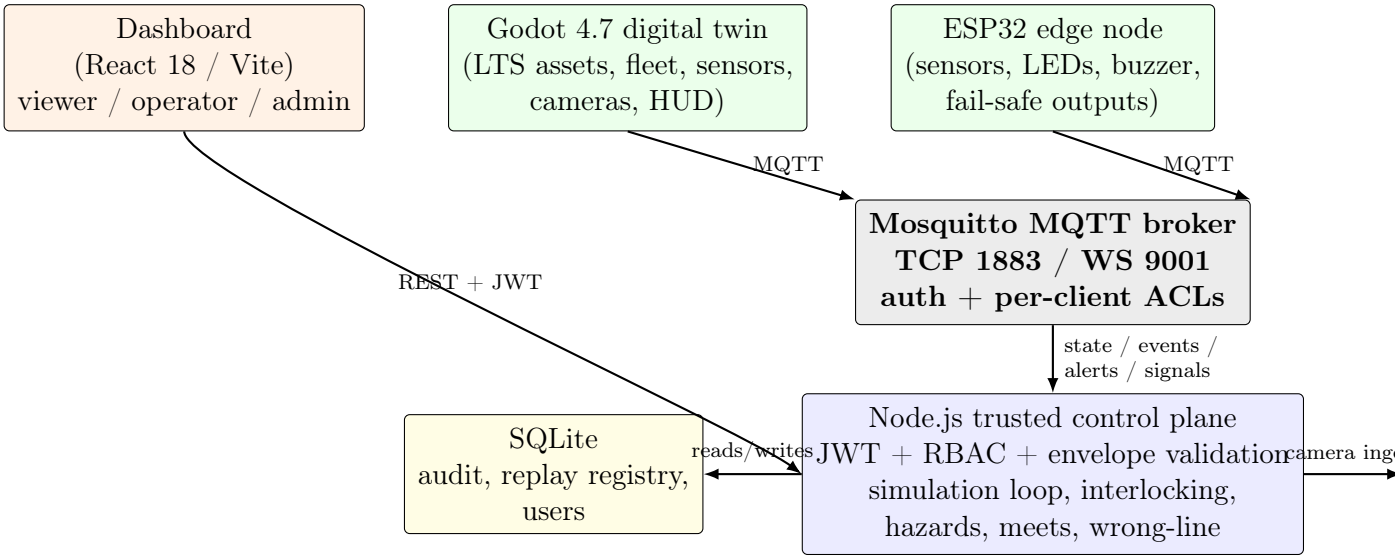


Figure 3.1: System block diagram: one trusted control plane, many clients.

The four cooperating surfaces are:

1. **Mosquitto broker** (TCP 1883, WebSocket 9001) with `allow_anonymous false`; credentials are generated by `scripts/mosquitto_setup.sh` from local environ-

ment variables; per-client ACLs in `mosquitto/acl_file` enforce the topic contract (Appendix **??**).

2. **Node.js backend** — authenticates users with bcrypt and JWT, enforces RBAC (viewer/operator/admin), validates command envelopes, blocks replays through a persistent SQLite `processed_requests` registry, writes an audit trail for every control attempt, runs the two-train simulation loop, owns block signalling, crossover interlocking, meet scheduling, the wrong-line state machine and the wildlife-hazard fusion — and is the sole MQTT publisher of commands, state, alerts and signals.

3. **Godot 4.7 twin** — full-scale double-track world with the LTS-derived asset fleet (Chapter **??**); reconciles backend state into train targets and signals, publishes track telemetry and wildlife evidence; also executes its own local fail-safe limit that can only *tighten* the backend target.

4. **React dashboard** — authenticated REST for control, MQTT-over-WebSocket for live updates; read-only on MQTT.

5. **ESP32 node** — authenticated IoT client restricted to telemetry publish + command subscribe on its own topics.

3.2 Data flow and message contract

Figure 3.2 traces the canonical control and observation paths. Two flows are deliberately asymmetric:

- **Control flows REST** (dashboard → backend only) so every command crosses the authenticated, audited, anti-replayed boundary once.
- **State flows MQTT** (backend → all clients, retained) so any late joiner sees the current truth without polling, and a lost client resynchronises from the retained snapshot on reconnect.

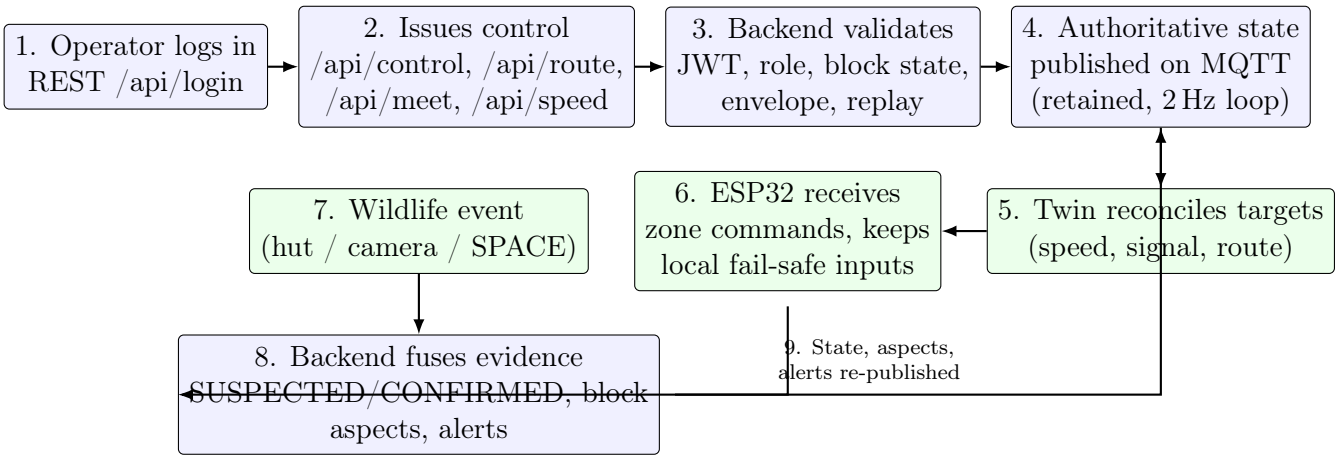


Figure 3.2: End-to-end workflow: control (steps 1–4) and wildlife observation (steps 5–9) as one closed loop.

3.3 Backend internal architecture

The backend is composed of small single-responsibility modules (`backend/src/`) composing a fixed-timestep simulation loop (`loop.js`) that ticks authoritative state and publishes snapshots at 2 Hz once the broker connection is up:

- `config.js` — fleet, blocks, lines, station, crossovers, meet parameters and wrong-line hold points (UP 1253.9822 m, DOWN 1228.8495 m lengths).
- `state.js` / `simulation.js` — authoritative trains T1 (UP, clockwise) and T3 (DOWN, anticlockwise), block occupancy, per-train speed dynamics and dwell logic.
- `signalling.js` / `routes.js` — per-block aspects, retained `railway/signal/{block}` topics, station crossover interlocking and route lifecycle.
- `meets.js` / `rng.js` — seeded chance-based meet scheduler and its deterministic PRNG.
- `wrong_line.js` — the operator-requested wrong-line run: APPROACH → CROSS_ENTRY → WRONG_LINE → CROSS_EXIT → CLEARANCE → DONE with automated opposing-train hold inside braking distance.
- `wildlife_commands.js` / `ingest.js` / `ml.js` — trackside and onboard evidence fusion, camera ingest (bearer token, `X-Captured-At`, duplicate-id, rate and size limits) and ML service calls.

- `auth.js` / `rbac.js` / `replay.js` / `audit.js` / `db.js` — JWT authentication, role checks, replay registry and audit logging backed by SQLite.
- `bus.js` / `mqtt.js` / `events.js` — MQTT connection, retained topic publication, WebSocket-ready event stream for the dashboard.
- `geometry.js` — shared UP/DOWN operational vs physical chainage conversion used by holds, huts and cameras.

3.4 Failure handling and fail-safety

Three deliberate, test-verified behaviours define the safety posture (Section **??**):

- **Restrictive fail-safe:** on MQTT loss the backend flags a restrictive state; the twin enforces a local limit; the ESP32 is restrictive at boot, during loss of fresh backend context, and for any active local hazard/fault input. Local inputs are debounced and can never be overridden remotely.
- **Recovery latch:** clearing a hazard is never enough; the backend latches `RECOVERY_REQUIRED` until an authorised operator/admin issues RESET with healthy sensors and no active hazard. RESET is further rejected while hazards or faults are active (per `IMPLEMENTATION.md`).
- **No command/telemetry replay:** envelopes carry timestamps (validated within ±5 min); IDs deduplicated in SQLite; ESP32 telemetry is non-retained, and stale, malformed, duplicate or wrong-zone telemetry is rejected. Firmware does not queue old telemetry or commands for replay after reconnect.

3.5 Topic contract summary

The complete per-client publish/subscribe matrix is in Appendix **??**. The contract's essence:

- backend publishes `railway/state`, `railway/events`, `railway/alerts`, `railway/zone*/signal`, `railway/commands/*`, `railway/speed/*`, `railway/security/events`, `railway/wildlife`, `railway/signal/{block}` (retained);
- godot publishes telemetry and wildlife observations for its zones; subscribes to commands, speeds, signals and state;

- dashboard subscribes to state, events and alerts — and publishes nothing;
- esp32 publishes only `railway/telemetry/esp32` (non-retained) and subscribes to `railway/commands/esp32`.

Captured figures of the twin surfaces these chapters describe are in Figures **??**–**??**.